ITAM SQL Structure Review

Source analyzed: `itam-imp.sql`

Scope and reading note

This document maps the tables from the SQL dump into a practical domain model and proposes an **ideal migration sequence** for rebuilding or refactoring the **inventory feature** in Laravel/MySQL.

Because the dump relies heavily on `*_id` naming conventions and indexes rather than explicit `FOREIGN KEY` constraints, the relationships below are **inferred relationships**, not guaranteed database-enforced relationships.

Scope adjustment for your implementation

This version intentionally keeps the **current platform layer** as-is and limits the rebuild plan to the **inventory feature**.

Current platform-owned tables in this app:

- `users` is a **first-party application table** that already exists
- auth uses Laravel defaults such as `password_reset_tokens`, `sessions`, and Sanctum `personal_access_tokens`
- RBAC is also **first-party and already separate** from inventory, using `roles`, `permissions`, `role_permission`, and `user_role`

That means:

- auth tables are **not part of the inventory migration plan**
- RBAC tables are **not part of the inventory migration plan**
- `users` is **not excluded from the data model**, but it is treated as an **existing upstream table** that inventory tables may reference
- any `user_id`, `created_by`, `assigned_to`, `manager_id`, or similar user references are still mapped, but they should point to the app's existing `users` table rather than a newly rebuilt inventory-owned user table

---

1. Domain table map

1.1 Core organization structure

Table	Purpose	Important columns	Likely references
`companies`	Company master	`id`, `created_by`	referenced by `assets`, `locations`, `licenses`, `accessories`, `components`, `consumables`, `departments`; `created_by` likely points to existing `users`
`locations`	Branch/site/location master	`company_id`, `parent_id`, `manager_id`, `created_by`	`companies`, self (`locations`), existing `users`, `assets`
`departments`	Department master	`company_id`, `location_id`, `manager_id`, `created_by`	`companies`, `locations`, existing `users`
`settings`	Global application config	`created_by`	existing `users`

Core relationship view

- `companies` 1 -> * `locations`
- `companies` 1 -> * `departments`
- `locations.parent_id -> locations.id`
- `locations.manager_id -> users.id` (existing app table)
- `departments.manager_id -> users.id` (existing app table)

1.2 Reference and master data

Table	Purpose	Important columns	Likely references
`categories`	Category master	`created_by`	`models`, `accessories`, `consumables`, `licenses`; `created_by` likely points to existing `users`
`manufacturers`	Manufacturer master	`created_by`	`models`, `accessories`, `components`, `consumables`, `licenses`; `created_by` likely points to existing `users`
`suppliers`	Supplier/vendor master	`created_by`	`assets`, `asset_maintenances`, `accessories`, `components`, `consumables`, `licenses`; `created_by` likely points to existing `users`
`status_labels`	Asset status master	`created_by`	`assets`; `created_by` likely points to existing `users`
`models`	Asset model master	`manufacturer_id`, `category_id`, `fieldset_id`, `created_by`	`manufacturers`, `categories`, `custom_fieldsets`, `assets`, existing `users`

> Laravel note: the table name `models` is valid, but the application class should not be named `App\Models\Model`. Use an explicit domain class name such as `AssetModel`.

1.3 Custom field subsystem

Table	Purpose	Important columns	Likely references
`custom_fields`	Custom field definitions	`created_by`	`custom_field_custom_fieldset`, `models_custom_fields`, existing `users`
`custom_fieldsets`	Fieldset/group for custom fields	`created_by`	`models`, `custom_field_custom_fieldset`, existing `users`
`custom_field_custom_fieldset`	Pivot: field <-> fieldset	`custom_field_id`, `custom_fieldset_id`	`custom_fields`, `custom_fieldsets`
`models_custom_fields`	Pivot: model <-> custom field default value	`asset_model_id`, `custom_field_id`	`models`, `custom_fields`

Custom field relationship view

- `custom_fields` * -> * `custom_fieldsets`
- `models` * -> * `custom_fields`
- `custom_fieldsets` 1 -> * `models`

1.4 Core inventory entities

Table	Purpose	Important columns	Likely references
`assets`	Main asset inventory table	`model_id`, `assigned_to`, `assigned_type`, `status_id`, `supplier_id`, `company_id`, `location_id`, `rtd_location_id`, `created_by`	`models`, existing `users`, `status_labels`, `suppliers`, `companies`, `locations`
`accessories`	Accessory stock master	`category_id`, `location_id`, `company_id`, `manufacturer_id`, `supplier_id`, `created_by`	`categories`, `locations`, `companies`, `manufacturers`, `suppliers`, existing `users`
`components`	Component stock master	`category_id`, `location_id`, `company_id`, `manufacturer_id`, `supplier_id`, `created_by`	`categories`, `locations`, `companies`, `manufacturers`, `suppliers`, existing `users`
`consumables`	Consumable stock master	`category_id`, `location_id`, `company_id`, `manufacturer_id`, `supplier_id`, `created_by`	`categories`, `locations`, `companies`, `manufacturers`, `suppliers`, existing `users`
`licenses`	License/software master	`supplier_id`, `company_id`, `manufacturer_id`, `category_id`, `created_by`	`suppliers`, `companies`, `manufacturers`, `categories`, existing `users`

Important note on `assets`

`assets` is the main operational table and currently mixes:

- core master data
- ownership and assignment state
- audit counters
- lifecycle dates
- many `_snipeit_*` spec columns

That makes it the most central table and also the hardest one to refactor safely.

1.5 Operational and transaction tables

Table	Purpose	Important columns	Likely references
`asset_maintenances`	Maintenance history	`asset_id`, `supplier_id`, `created_by`	`assets`, `suppliers`, existing `users`
`asset_uploads`	Asset files/documents	`user_id`, `asset_id`	existing `users`, `assets`
`action_logs`	Generic audit/activity log	`created_by`, `target_type`, `target_id`, `item_type`, `item_id`, `location_id`, `company_id`, `accepted_id`	polymorphic targets/items, existing `users`, `locations`, `companies`
`asset_logs`	Asset action log	`user_id`, `asset_id`, `checkedout_to`, `location_id`, `accessory_id`, `consumable_id`, `component_id`, `accepted_id`	existing `users`, `assets`, `locations`, `accessories`, `consumables`, `components`
`checkout_acceptances`	Signature/acceptance record	`checkoutable_type`, `checkoutable_id`, `assigned_to_id`	polymorphic checkout target, existing `users`
`checkout_requests`	Request for a requestable item	`user_id`, `requestable_type`, `requestable_id`	existing `users`, polymorphic requestable entity
`requested_assets`	Legacy or asset-specific request table	`asset_id`, `user_id`	`assets`, existing `users`
`requests`	Legacy generic request table	`asset_id`, `user_id`	`assets`, existing `users`
`report_templates`	Saved reports	`created_by`	existing `users`
`imports`	Import session/history	`created_by`	existing `users`

Operational relationship view

- `assets` 1 -> * `asset_maintenances`
- `assets` 1 -> * `asset_uploads`
- `assets` 1 -> * `requested_assets`
- `action_logs` and `asset_logs` are audit/history tables

1.6 Inventory movement and assignment tables

Accessories

Table	Purpose	Important columns	Likely references
`accessories_checkout`	Accessory assignment/checkout	`created_by`, `accessory_id`, `assigned_to`, `assigned_type`	existing `users`, `accessories`, polymorphic assignee

Components

Table	Purpose	Important columns	Likely references
`components_assets`	Pivot: component attached to asset	`created_by`, `component_id`, `asset_id`	existing `users`, `components`, `assets`

Consumables

Table	Purpose	Important columns	Likely references
`consumables_users`	Consumable distribution/assignment	`created_by`, `consumable_id`, `assigned_to`	existing `users`, `consumables`

Licenses

Table	Purpose	Important columns	Likely references
`license_seats`	License seat assignment	`license_id`, `assigned_to`, `asset_id`, `created_by`	`licenses`, existing `users`, `assets`

1.7 Kits and bundles

Table	Purpose	Important columns	Likely references
`kits`	Bundle/kit master	`created_by`	`kits_accessories`, `kits_consumables`, `kits_licenses`, `kits_models`, existing `users`
`kits_accessories`	Pivot: kit <-> accessory	`kit_id`, `accessory_id`, `created_by`	`kits`, `accessories`, existing `users`
`kits_consumables`	Pivot: kit <-> consumable	`kit_id`, `consumable_id`, `created_by`	`kits`, `consumables`, existing `users`
`kits_licenses`	Pivot: kit <-> license	`kit_id`, `license_id`, `created_by`	`kits`, `licenses`, existing `users`
`kits_models`	Pivot: kit <-> model	`kit_id`, `model_id`, `created_by`	`kits`, `models`, existing `users`

2. Most important inferred joins

From	To	Join
`assets`	`models`	`assets.model_id = models.id`
`assets`	`status_labels`	`assets.status_id = status_labels.id`
`assets`	`suppliers`	`assets.supplier_id = suppliers.id`
`assets`	`companies`	`assets.company_id = companies.id`
`assets`	`locations`	`assets.location_id = locations.id`
`assets`	`locations`	`assets.rtd_location_id = locations.id`
`assets`	`users`	`assets.assigned_to = users.id` when `assigned_type = users`
`models`	`manufacturers`	`models.manufacturer_id = manufacturers.id`
`models`	`categories`	`models.category_id = categories.id`
`models`	`custom_fieldsets`	`models.fieldset_id = custom_fieldsets.id`
`locations`	`locations`	`locations.parent_id = locations.id`
`locations`	`users`	`locations.manager_id = users.id` (existing app table)
`departments`	`users`	`departments.manager_id = users.id` (existing app table)
`license_seats`	`licenses`	`license_seats.license_id = licenses.id`
`license_seats`	`users`	`license_seats.assigned_to = users.id`
`license_seats`	`assets`	`license_seats.asset_id = assets.id`
`components_assets`	`components`	`components_assets.component_id = components.id`
`components_assets`	`assets`	`components_assets.asset_id = assets.id`
`asset_maintenances`	`assets`	`asset_maintenances.asset_id = assets.id`
`asset_maintenances`	`suppliers`	`asset_maintenances.supplier_id = suppliers.id`

---

3. Where the dependency cycles exist

These columns create dependency cycles or late-binding relationships and should **not** be the first constraints you add:

- any `created_by` column -> depends on existing `users`
- any `user_id` actor column -> depends on existing `users`
- `users` itself already exists in the app and is outside the inventory rebuild scope
- `locations.parent_id` -> self-reference to `locations`
- `locations.manager_id` -> depends on existing `users`
- `departments.manager_id` -> depends on existing `users`
- `assigned_type` + `assigned_to` -> polymorphic, usually **do not** FK directly
- `requestable_type` + `requestable_id` -> polymorphic
- `checkoutable_type` + `checkoutable_id` -> polymorphic
- `target_type` + `target_id` -> polymorphic
- `item_type` + `item_id` -> polymorphic

Design implication

The cleanest migration strategy is:

1. create inventory tables first
2. add direct foreign keys where dependency is clear
3. add user-linked constraints only after the inventory tables are in place and aligned with the existing `users` table
4. add self-referencing constraints in a later patch migration
5. keep polymorphic columns indexed, not foreign keyed

---

4. Ideal migration sequence

4.1 Principles used for the sequence

This is the most practical order if you want a clean Laravel migration plan:

- start with independent master tables
- then create organizational tables
- then create reference masters for inventory
- then inventory masters and core entities
- then pivots and operational tables
- finally, add user-linked and self-referencing foreign keys in patch migrations

Because `users`, auth, and RBAC already exist in the application layer, this sequence assumes **those platform tables remain untouched** while inventory tables are rebuilt around them.

4.2 Recommended migration order

Phase A - foundational masters

1. `create_companies_table`
2. `create_categories_table`
3. `create_manufacturers_table`
4. `create_suppliers_table`
5. `create_status_labels_table`
6. `create_custom_fields_table`
7. `create_custom_fieldsets_table`
8. `create_kits_table`

Phase B - organizational structure

9. `create_locations_table`
10. `create_departments_table`
11. `create_settings_table`

Phase C - model and metadata layer

12. `create_models_table`
13. `create_custom_field_custom_fieldset_table`
14. `create_models_custom_fields_table`

Phase D - core inventory entities

15. `create_assets_table`
16. `create_accessories_table`
17. `create_components_table`
18. `create_consumables_table`
19. `create_licenses_table`

Phase E - bundle/pivot inventory tables

20. `create_kits_accessories_table`
21. `create_kits_consumables_table`
22. `create_kits_licenses_table`
23. `create_kits_models_table`
24. `create_components_assets_table`
25. `create_license_seats_table`

Phase F - checkout, distribution, and maintenance flows

26. `create_accessories_checkout_table`
27. `create_consumables_users_table`
28. `create_asset_maintenances_table`
29. `create_asset_uploads_table`
30. `create_checkout_acceptances_table`
31. `create_checkout_requests_table`
32. `create_requested_assets_table`
33. `create_requests_table`

Phase G - logs, reporting, and imports

34. `create_action_logs_table`
35. `create_asset_logs_table`
36. `create_imports_table`
37. `create_report_templates_table`

> Note: `migrations` is not listed because Laravel handles it automatically. Auth and RBAC tables are also intentionally excluded because they already exist and are not part of the inventory rebuild target.

---

5. Ideal foreign key rollout sequence

The table creation order above is not exactly the same as the **constraint** order.

5.1 Add direct foreign keys during or immediately after base table creation

Safe early foreign keys

- `locations.company_id -> companies.id`
- `departments.company_id -> companies.id`
- `departments.location_id -> locations.id`
- `models.manufacturer_id -> manufacturers.id`
- `models.category_id -> categories.id`
- `models.fieldset_id -> custom_fieldsets.id`
- `models_custom_fields.asset_model_id -> models.id`
- `models_custom_fields.custom_field_id -> custom_fields.id`
- `custom_field_custom_fieldset.custom_field_id -> custom_fields.id`
- `custom_field_custom_fieldset.custom_fieldset_id -> custom_fieldsets.id`
- `assets.model_id -> models.id`
- `assets.status_id -> status_labels.id`
- `assets.supplier_id -> suppliers.id`
- `assets.company_id -> companies.id`
- `assets.location_id -> locations.id`
- `assets.rtd_location_id -> locations.id`
- `accessories.category_id -> categories.id`
- `accessories.location_id -> locations.id`
- `accessories.company_id -> companies.id`
- `accessories.manufacturer_id -> manufacturers.id`
- `accessories.supplier_id -> suppliers.id`
- `components.category_id -> categories.id`
- `components.location_id -> locations.id`
- `components.company_id -> companies.id`
- `components.manufacturer_id -> manufacturers.id`
- `components.supplier_id -> suppliers.id`
- `consumables.category_id -> categories.id`
- `consumables.location_id -> locations.id`
- `consumables.company_id -> companies.id`
- `consumables.manufacturer_id -> manufacturers.id`
- `consumables.supplier_id -> suppliers.id`
- `licenses.supplier_id -> suppliers.id`
- `licenses.company_id -> companies.id`
- `licenses.manufacturer_id -> manufacturers.id`

- `licenses.category_id -> categories.id`
- `license_seats.license_id -> licenses.id`
- `license_seats.asset_id -> assets.id`
- `components_assets.component_id -> components.id`
- `components_assets.asset_id -> assets.id`
- `asset_maintenances.asset_id -> assets.id`
- `asset_maintenances.supplier_id -> suppliers.id`
- `asset_uploads.asset_id -> assets.id`
- `requested_assets.asset_id -> assets.id`
- `requests.asset_id -> assets.id`
- `kits_accessories.kit_id -> kits.id`
- `kits_accessories.accessory_id -> accessories.id`
- `kits_consumables.kit_id -> kits.id`
- `kits_consumables.consumable_id -> consumables.id`
- `kits_licenses.kit_id -> kits.id`
- `kits_licenses.license_id -> licenses.id`
- `kits_models.kit_id -> kits.id`
- `kits_models.model_id -> models.id`

5.2 Add late foreign keys in a dedicated patch migration

These are better added after the main tables exist, the current app `users` table is confirmed, and data is cleaned:

- `locations.parent_id -> locations.id`
- `locations.manager_id -> users.id`
- `departments.manager_id -> users.id`
- all `created_by -> users.id` relationships where data quality is trustworthy
- `assets.created_by -> users.id`
- `asset_uploads.user_id -> users.id`
- `asset_maintenances.created_by -> users.id`
- `imports.created_by -> users.id`
- `report_templates.created_by -> users.id`
- `requested_assets.user_id -> users.id`
- `requests.user_id -> users.id`
- `checkout_requests.user_id -> users.id`
- `license_seats.assigned_to -> users.id`
- other nullable operational actor fields if historical data is already clean

5.3 Keep polymorphic relations non-FK

These should usually remain indexed only:

- `assets.assigned_type` + `assets.assigned_to`
- `accessories_checkout.assigned_type` + `assigned_to`
- `checkout_requests.requestable_type` + `requestable_id`
- `checkout_acceptances.checkoutable_type` + `checkoutable_id`
- `action_logs.target_type` + `target_id`
- `action_logs.item_type` + `item_id`

Reason: they are generic polymorphic references, not single-target relations.

5.4 Add uniqueness rules for pivots and identity columns

Foreign keys alone are not enough. The inventory rebuild should also define uniqueness rules explicitly:

- use composite primary keys or composite unique indexes for pure pivots such as `custom_field_custom_fieldset`, `components_assets`, `kits_accessories`, `kits_consumables`, `kits_licenses`, and `kits_models`
- if `models_custom_fields` stores only one default value per model-field pair, enforce uniqueness on (`asset_model_id`, `custom_field_id`)
- decide whether movement tables such as `accessories_checkout`, `consumables_users`, and `license_seats` are historical logs or current-state pivots before adding uniqueness
- decide explicit uniqueness for business identifiers such as `assets.asset_tag`, and possibly `assets.serial` if the business treats serial numbers as globally unique

---

6. Recommended cleanup before enforcing foreign keys

Before applying strict constraints, clean these areas first:

1. **Standardize key types**
    - Many tables use `int unsigned`, some use plain `int`, some use `bigint unsigned`.
    - Standardize PK/FK pairs first.
2. **Normalize datetime strategy**
    - `deleted_at` is mixed between `timestamp` and `datetime`.
    - Standardize for Laravel consistency.
3. **Review request-table overlap**
    - `requests`, `requested_assets`, and `checkout_requests` likely overlap.
    - For an inventory-focused rebuild, choose one canonical request workflow and treat the others as legacy compatibility tables unless there is a proven separate use case.
4. **Audit user-linked columns**
    - Do not add user foreign keys until the existing app `users` table shape is confirmed and orphan IDs are resolved.
5. **Decide uniqueness rules**
    - Especially for `assets.asset_tag`, and maybe `assets.serial` depending on business rules.

---

7. Practical recommendation if this is rebuilt in Laravel

A strong implementation approach would be:

- **Batch 1:** create all inventory-domain tables with basic indexes only
- **Batch 2:** add safe direct foreign keys to non-user masters
- **Batch 3:** align inventory user references with the existing app `users` table
- **Batch 4:** clean legacy/orphan user references
- **Batch 5:** add self-referencing and user-linked foreign keys
- **Batch 6:** add unique constraints and stricter checks

This is safer than trying to enforce everything in the first migration pass.

If you implement this in Laravel:

- use Artisan first for framework files, for example `php artisan make:migration create_assets_table --create=assets --no-interaction`
- keep inventory migrations separate from auth and RBAC migrations
- prefer `foreignId()->constrained()` only where the target table is part of the inventory scope and already settled
- use patch migrations for late user-linked and self-referencing constraints

---

8. Bottom line

The schema is usable and functionally rich, but the most ideal migration strategy is **dependency-first, constraint-second**.

For your current app, that means:

- this migration plan only covers the **inventory domain**
- auth and RBAC are intentionally excluded because they already exist in the app layer
- `users` remains a **first-party app table**, but inventory does not rebuild it
- table creation order should follow master data -> org structure -> inventory masters -> core inventory -> pivots/transactions -> logs/support tables
- user-linked foreign keys should be staged after inventory tables are created and checked against the existing `users` table
- polymorphic links should stay non-FK
- self-references and `created_by` should be patched in later